Mini Transformer Benchmark

Zin Thein Theint Htun Lwin
67070701610

1. Introduction

The Transformer architecture has become a fundamental model in sequence processing tasks due to its ability to capture long-range dependencies through attention mechanisms. This project aims to implement a mini Transformer encoder from scratch using PyTorch in order to gain a deeper understanding of its core components.

Unlike standard implementations, this work avoids the use of prebuilt Transformer modules, requiring manual implementation of attention, multi-head mechanisms, and encoder blocks. Additionally, a small benchmark is conducted to evaluate the impact of architectural choices such as positional encoding, number of attention heads, and number of layers.

2. Task Description

The task is a synthetic sequence classification problem. Given a sequence of discrete tokens with padding, the model must predict whether:

The first non-padding token appears again in the second half of the non-padding sequence.

Sequence: [A, C, B, D, A, PAD, PAD]

- First token: A
- Non-padding sequence: [A, C, B, D, A]
- Second half: [D, A]
- Since A appears in the second half, the label is 1

This task requires the model to understand sequence order, ignore padding, and capture relationships between tokens.

3. Dataset

The dataset consists of sequences composed of tokens from a fixed vocabulary:

Token	ID
PAD	0
A	1
B	2
C	3
D	4

Each sequence has:

- Variable length between 6 and 20
- Padding applied to reach a fixed length of 20
- An attention mask indicating valid (non-padding) tokens

The label is computed by splitting the non-padding sequence into two halves and checking whether the first token appears in the second half.

4. Model Architecture

The model follows a standard Transformer encoder pipeline, implemented manually:

4.1 Token Embedding

Input tokens are mapped to dense vectors using an embedding layer with a dedicated padding index.

4.2 Positional Encoding

Sinusoidal positional encoding is added to embeddings to incorporate sequence order information. This component is optionally disabled in some experiments.

4.3 Scaled Dot-Product Attention

Attention is computed using:

$$Attention\ (Q, K, V) = \text{softmax}\ (\frac{QK^T}{\sqrt{d_k}})V$$

Padding masks are applied to prevent attention over padded positions.

4.4 Multi-Head Self-Attention

The model uses multiple attention heads to capture different relationships in the sequence. Outputs from all heads are concatenated and linearly projected.

4.5 Feed-Forward Network

A position-wise feed-forward network consisting of two linear layers and a ReLU activation is applied.

4.6 Encoder Block

Each encoder block consists of:

- Multi-head attention + residual connection + layer normalization
- Feed-forward network + residual connection + layer normalization

4.7 Pooling and Classification

Masked mean pooling is applied to obtain a fixed-size representation, followed by a linear classification head for binary prediction.

5. Experimental Setup

The model was trained using the following hyperparameters:

Parameter	Value
Embedding dimension	64
Feed-forward dimension	128
Batch size	32
Learning rate	0.001
Epochs	15 (Training), 10 & 20 (benchmark experiments)
Optimizer	Adam
Loss function	Binary Cross-Entropy with Logits (BCEWithLogitsLoss)

Different epoch settings were used for different purposes. The standalone training script uses 15 epochs, while the benchmark experiments were conducted using both 10 and 20 epochs to compare convergence behavior and performance. Model selection is based on validation loss.

6. Model Configurations

Four model variants were evaluated:

Model	Positional Encoding	Heads	Layers
A	Yes	1	1
B	Yes	4	1
C	No	4	1
D	Yes	4	2

7. Results

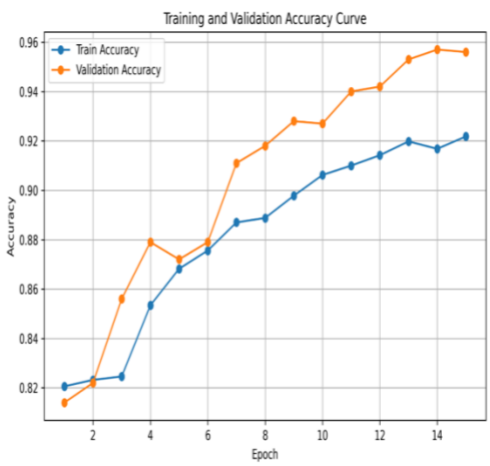
7.1 Results (20 Epochs)

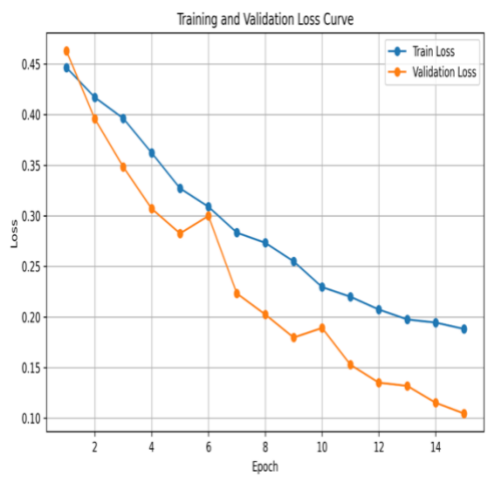
```
=== Running Model_A ===

=== Running Model_B ===

=== Running Model_C ===

=== Running Model_D ===

=== Benchmark Results ===
     Model  PositionalEncoding  Heads  Layers  Best_Epoch  Val_Loss  Val_Acc  Test_Loss  Test_Acc  Params  Time_min
0  Model_A                True      1       1          18    0.2199    0.916     0.2962     0.889   33857      0.33
1  Model_B                True      4       1          19    0.0732    0.971     0.0836     0.969   33857      0.51
2  Model_C               False      4       1          17    0.4795    0.814     0.4672     0.823   33857      0.48
3  Model_D                True      4       2          20    0.0324    0.992     0.0415     0.994   67329      0.89
```

7.2 Results (10 Epochs)

```
=== Running Model_A ===

=== Running Model_B ===

=== Running Model_C ===

=== Running Model_D ===

=== Benchmark Results ===
     Model  PositionalEncoding  Heads  Layers  Best_Epoch  Val_Loss  Val_Acc  Test_Loss  Test_Acc  Params  Time_min
0  Model_A                True      1       1           9    0.2685    0.896     0.3407     0.879   33857      0.15
1  Model_B                True      4       1           9    0.1797    0.928     0.2185     0.913   33857      0.20
2  Model_C               False      4       1           5    0.4803    0.814     0.4675     0.823   33857      0.19
3  Model_D                True      4       2           9    0.0531    0.985     0.0835     0.969   67329      0.39
```

7.3 Training Behavior





Training curves show:

- Gradual increase in accuracy
- Consistent decrease in loss
- No significant overfitting observed
- Stable convergence across models

8. Discussion

8.1 Effect of Positional Encoding

Removing positional encoding significantly reduces performance. Model C performs much worse than Model B, confirming that positional information is essential for sequence-based tasks.

8.2 Effect of Attention Heads

Increasing the number of attention heads improves performance. Model B outperforms Model A, indicating that multiple heads allow the model to capture richer relationships.

8.3 Effect of Depth

Adding an additional encoder layer (Model D) results in further performance improvement. However, the gain is smaller compared to other factors, suggesting diminishing returns for deeper models on simple tasks.

8.4 Effect of Training Duration

The results show that increasing the number of training epochs from 10 to 20 leads to consistent performance improvements across all models. This indicates that the models had not fully converged at 10 epochs. However, no overfitting was observed at 20 epochs, suggesting that the training duration is appropriate for this task.

8.5 Best Model

Model D achieves the highest performance, with a test accuracy of 99.4%. This demonstrates that combining positional encoding, multiple heads, and deeper architecture yields the best results.

9. Conclusion

This project demonstrates that key components of the Transformer architecture play distinct roles in model performance:

- Positional encoding is essential for sequence understanding
- Multi-head attention significantly improves performance
- Increasing depth provides moderate additional gains

Even in a simplified setting, the Transformer architecture proves to be highly effective for sequence classification tasks.

10. Limitations and Future Work

This work is limited by the simplicity of the synthetic dataset, which does not fully represent real-world sequence complexity. The task focuses on a specific pattern detection problem and does not require deep semantic understanding.

Additionally, the model size and architecture are relatively small, which limits its capacity compared to full-scale Transformer models.

For future work, the model can be evaluated on more complex or real-world datasets. Further improvements could include experimenting with different pooling strategies, larger embedding dimensions, and deeper architectures. Exploring regularization techniques and alternative optimization strategies may also improve performance.